

Sprint Documentation		
Sprint 2	Start Date: 13/10/25	Final Date: 20/10/25
Projetc Name:	civitas-backend	
Ano: 4º	Análise e Desenvolvimento de Sistemas – MAS	
Team Members		
Felipe Pacheco Bianchini		
Wendell Alves Nascimento		
Giovanni da Silva Nascimento		

Sprint Backlog			
Task#	Description	Start Date	Final date
27	Criação do CRUD de Auditoria	13/10/25	20/10/25

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
27	Implementar os métodos padrão do CRUD para a entidade Auditoria, contemplando as seguintes operações: -Cadastrar -Alterar -Inativar e ativar registro -Listar	Felipe Pacheco Bianchini, Wendell Alves Nascimento, Giovanni da Silva Nascimento	In Progress	10	10

Sprint Description

Implementar os métodos padrão do CRUD para a entidade Auditoria, contemplando as seguintes operações:

-Cadastrar

-Alterar

-Inativar e ativar registro

-Listar

RELATÓRIO DA SPRINT 2 - TASK 27: CRUD DE AUDITORIA

1. INTRODUÇÃO

Este relatório documenta a implementação do sistema CRUD para a entidade Auditoria no projeto Civitas Backend, desenvolvido durante a Sprint 2.

2. OBJETIVO

Criar um sistema completo para gerenciar registros de auditoria, permitindo cadastrar, alterar, inativar, ativar e listar operações realizadas no sistema, com vínculo ao usuário responsável.

3. O QUE FOI DESENVOLVIDO

3.1 Estrutura da Entidade Auditoria

A entidade Auditoria possui os seguintes campos:

- id: Identificador único
- data: Data da operação
- hora: Horário da operação
- operacao: Tipo de operação (INSERT, UPDATE, DELETE)
- nomeEntidade: Nome da tabela/entidade auditada
- valoresAtingidos: Valores antigos (formato JSON)
- novosValores: Valores novos (formato JSON)
- situacao: Status do registro (Ativo/Inativo)
- userId: Identificador do usuário responsável
- usuario: Dados do usuário relacionado

3.2 Componentes Implementados

3.2.1 Model e DTO

Criação da classe Auditoria e seu respectivo DTO para transferência de dados entre as camadas do sistema.

3.2.2 Builder

Implementação do padrão Builder para converter dados entre Model e DTO.

3.2.3 Repository

Criação do repositório com métodos para acesso ao banco de dados:

- Métodos básicos herdados do GenericRepository
- Busca por usuário
- Busca por entidade
- Busca por tipo de operação

3.2.4 Service

Implementação das regras de negócio e validações:

- Cadastro com validações
- Alteração de registros
- Inativação de registros (soft delete)
- Ativação de registros
- Listagem e buscas específicas

3.2.5 Controller

Criação dos endpoints da API REST:

- POST /api/Auditoria - Cadastrar
- PUT /api/Auditoria/{id} - Alterar
- DELETE /api/Auditoria/inativar/{id} - Inativar
- PATCH /api/Auditoria/ativar/{id} - Ativar
- GET /api/Auditoria - Listar todos
- GET /api/Auditoria/{id} - Buscar por ID
- GET /api/Auditoria/usuario/{userId} - Buscar por usuário
- GET /api/Auditoria/entidade/{nomeEntidade} - Buscar por entidade
- GET /api/Auditoria/operacao/{operacao} - Buscar por operação

3.2.6 Banco de Dados

Criação da tabela auditoria no banco de dados PostgreSQL com relacionamento à tabela usuario.

Comandos executados:

```
dotnet ef migrations add AddAuditoriaTables
```

```
dotnet ef database update
```

4. EVIDÊNCIAS

 Swagger
powered by SMARTBEAR

Select a definition Civitas API v1

Civitas.WebAPI1.0OAS 3.0

/swagger/v1/swagger.json

Auditoria

GET/api/Auditoria

POST/api/Auditoria

GET/api/Auditoria/{id}

PUT/api/Auditoria/{id}

DELETE/api/Auditoria/{id}

GET/api/Auditoria/GetByUsuarioId/{usuarioId}

GET/api/Auditoria/GetByEntidade

GET/api/Auditoria/GetByOperacao

PATCH/api/Auditoria/{id}/alterar-situacao

Auditoria

GET/api/Auditoria

Parameters

No parameters

ExecuteClear

Responses

Curl

curl -X 'GET' \ 'http://localhost:5210/api/Auditoria' \ -H 'accept: */*'

Request URL

http://localhost:5210/api/Auditoria

Server response

CodeDetails

200

Response body

{ "code": 1, "message": "Lista de auditorias obtida com sucesso", "data": [] }

Download

Response headers

content-type: application/json; charset=utf-8 date: Fri, 17 Oct 2025 13:48:32 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description	Links
200	OK	No links

Code

Details

200

Response body

```
{  "code": 1,  "message": "Auditoria cadastrada com sucesso",  "data": {    "id": 0,    "data": "2025-10-17",    "hora": "10:45:00",    "operacao": "INSERT",    "nomeEntidade": "Produto",    "valoresAtingidos": "{}",    "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }",    "situacao": 1,    "usuarioId": 0,    "usuario": {      "id": 0,      "cpf": "12345678900",      "nome": "João da Silva",      "rg": "123456789",      "logradouro": "Rua das Flores",      "numero": "123",      "cidade": "São Paulo",      "estado": "SP",      "cep": "01000000",      "bairro": "Centro",      "email": "joao.silva@example.com",      "senha": "uma_senha_forte_123",      "matricula": "987654",      "situacao": 1,      "tipoUsuario": 1    }  }
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri, 17 Oct 2025 13:52:35 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

GET	/api/Auditoria/{id}								
Parameters Cancel <table><tr><th>Name</th><th>Description</th></tr><tr><td>id required</td><td></td></tr><tr><td>integer(\$int32)</td><td>1</td></tr><tr><td>(path)</td><td></td></tr></table> Execute Clear		Name	Description	id required		integer(\$int32)	1	(path)	
Name	Description								
id required									
integer(\$int32)	1								
(path)									
Responses Curl <pre>curl -X 'GET' \ 'http://localhost:5210/api/Auditoria/1' \ -H 'accept: */*'</pre> Request URL http://localhost:5210/api/Auditoria/1 Server response <table><tr><th>Code</th><th>Details</th></tr><tr><td>200</td><td> Response body <pre>{ "code": 1, "message": "Auditoria obtida com sucesso", "data": { "id": 1, "data": "2025-10-17", "hora": "10:45:00", "operacao": "INSERT", "nomeEntidade": "Produto", "valoresAtingidos": "{}", "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }", "situacao": 1, "usuarioId": 1, "usuario": null } }</pre> Download Response headers </td></tr></table>		Code	Details	200	Response body <pre>{ "code": 1, "message": "Auditoria obtida com sucesso", "data": { "id": 1, "data": "2025-10-17", "hora": "10:45:00", "operacao": "INSERT", "nomeEntidade": "Produto", "valoresAtingidos": "{}", "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }", "situacao": 1, "usuarioId": 1, "usuario": null } }</pre> Download Response headers				
Code	Details								
200	Response body <pre>{ "code": 1, "message": "Auditoria obtida com sucesso", "data": { "id": 1, "data": "2025-10-17", "hora": "10:45:00", "operacao": "INSERT", "nomeEntidade": "Produto", "valoresAtingidos": "{}", "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }", "situacao": 1, "usuarioId": 1, "usuario": null } }</pre> Download Response headers								

GET

/api/Auditoria/GetByEntidade

Parameters

Cancel

Name	Description
nomeEntidade	
string	Produto
(query)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  
'http://localhost:5210/api/Auditoria/GetByEntidade?nomeEntidade=Produto' \  
-H 'accept: */*'
```

Request URL

http://localhost:5210/api/Auditoria/GetByEntidade?nomeEntidade=Produto

Server response

Code	Details
200	Response body <pre>{ "code": 1, "message": "Auditorias da entidade listadas com sucesso", "data": [{ "id": 1, "data": "2025-10-17", "hora": "10:45:00", "operacao": "INSERIR", "nomeEntidade": "Produto", "valoresAtingidos": "{}", "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }", "situacao": 1, "usuarioId": 2, "usuario": { "id": 2, "cpf": "12345678900", "nome": "João da Silva",</pre>

GET

/api/Auditoria/GetByOperacao

^

Parameters

Cancel

Name	Description
operacao	
string	INSERT
(query)	

Execute

Clear

Responses

Curl

curl -X 'GET' \ 'http://localhost:5210/api/Auditoria/GetByOperacao?operacao=INSERT' \ -H 'accept: */*'

Request URL

http://localhost:5210/api/Auditoria/GetByOperacao?operacao=INSERT

Server response

Code	Details
200	Response body <pre>{ "code": 1, "message": "Auditorias da opera\u00e7\u00e3o listadas com sucesso", "data": [{ "id": 1, "data": "2025-10-17", "hora": "10:45:00", "operacao": "INSERT", "nomeEntidade": "Produto", "valoresAtingidos": "{}", "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }", "situacao": 1, "usuarioId": 2, "usuario": { "id": 2, "cpf": "12345678900", "nome": "Jo\u00e3o da Silva", "rg": "123456789", "logradouro": "Rua das Flores", </pre>

PATCH

/api/Auditoria/{id}/alterar-situacao

^

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Curl

curl -X 'PATCH' \ 'http://localhost:5210/api/Auditoria/1/alterar-situacao' \ -H 'accept: */*'

Request URL

http://localhost:5210/api/Auditoria/1/alterar-situacao

Server response

Code	Details
200	Response body <pre>{ "code": 1, "message": "Situa\u00e7\u00e3o alterada para INATIVO com sucesso", "data": { "id": 1, "situacao": "INATIVO" } }</pre> Download

Response headers

Code

Details

200

Response body

```
{
  "code": 1,
  "message": "Auditoria atualizada com sucesso",
  "data": {
    "id": 1,
    "data": "2025-10-17",
    "hora": "11:30:00",
    "operacao": "UPDATE",
    "nomeEntidade": "Produto",
    "valoresAtingidos": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 100 }",
    "novosValores": "{ \"nome\": \"Caneta Azul\", \"quantidade\": 150, \"preco\": 2.50 }",
    "situacao": 1,
    "usuarioId": 1,
    "usuario": null
  }
}
```

 Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri, 17 Oct 2025 14:00:09 GMT
server: Kestrel
transfer-encoding: chunked
```

DELETE

/api/Auditoria/{id}

^

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:5210/api/Auditoria/1' \
-H 'accept: */*'
```



Request URL

http://localhost:5210/api/Auditoria/1

Server response

Code	Details
200	Response body <pre>{ "code": 1, "message": "Auditoria excluída com sucesso", "data": null }</pre>  Download

Response headers

5. PONTOS POSITIVOS

5.1 Padrão de Código

O código seguiu o mesmo padrão das outras entidades do projeto (Usuário, Fornecedor, Secretaria), facilitando a manutenção.

5.2 Reutilização

Foi aproveitado o GenericRepository existente, evitando código duplicado e acelerando o desenvolvimento.

5.3 Relacionamento com Usuário

O vínculo entre Auditoria e Usuário foi implementado corretamente, permitindo rastrear quem realizou cada operação.

5.4 Validações

Foram implementadas validações para garantir que todos os dados obrigatórios sejam preenchidos corretamente.

5.5 Buscas Específicas

Além do CRUD básico, foram criados métodos de busca por usuário, entidade e tipo de operação, facilitando consultas futuras.

6. LIMITAÇÕES

6.1 Auditoria Automática

A auditoria não é automática. É necessário chamar os endpoints manualmente para registrar operações. Isso garante controle total sobre o que é auditado.

6.2 Paginação

As listagens não possuem paginação. Caso o volume de dados cresça muito, será necessário implementar em versões futuras.

6.3 Filtros Combinados

Os filtros funcionam individualmente (por usuário OU por entidade OU por operação). Filtros combinados podem ser adicionados futuramente se necessário.

6.4 Relatórios

Não há funcionalidade de exportar dados para PDF ou Excel. Esta funcionalidade será desenvolvida em outra task específica.

7. TESTES REALIZADOS

7.1 Teste de Cadastro

Teste: Cadastrar uma nova auditoria

Resultado: Aprovado - Registro criado com sucesso

7.2 Teste de Validação

Teste: Tentar cadastrar sem preencher campos obrigatórios

Resultado: Aprovado - Sistema retornou erro informando campos faltantes

7.3 Teste de Alteração

Teste: Alterar dados de uma auditoria existente

Resultado: Aprovado - Dados atualizados corretamente

7.4 Teste de Listagem

Teste: Listar todas as auditorias

Resultado: Aprovado - Lista retornada com dados do usuário relacionado

7.5 Teste de Busca por Usuário

Teste: Buscar auditorias de um usuário específico

Resultado: Aprovado - Filtro funcionou corretamente

7.6 Teste de Busca por Entidade

Teste: Buscar auditorias de uma entidade específica (ex: Produto)

Resultado: Aprovado - Busca retornou resultados corretos

7.7 Teste de Inativação

Teste: Inativar uma auditoria

Resultado: Aprovado - Registro foi marcado como inativo sem ser deletado

7.8 Teste de Ativação

Teste: Reativar uma auditoria inativa

Resultado: Aprovado - Registro foi reativado com sucesso

7.9 Teste de Integridade

Teste: Tentar cadastrar auditoria com usuário inexistente

Resultado: Aprovado - Sistema impediu a operação mantendo integridade dos dados

7.10 Teste de Performance

Teste: Verificar se as consultas ao banco estão otimizadas

Resultado: Aprovado - Queries geradas de forma eficiente

8. CONCLUSÃO

O CRUD de Auditoria foi implementado com sucesso, atendendo todos os requisitos da Task 27. O sistema permite:

- Cadastrar registros de auditoria vinculados a usuários
- Alterar informações quando necessário
- Inativar registros mantendo histórico
- Reativar registros quando necessário
- Listar e buscar auditorias de diversas formas

Todas as funcionalidades foram testadas e validadas. O código está documentado, segue os padrões do projeto e está pronto para uso.

Assinaturas